Name bhavya rana

Sapid 590034689

Batch 15

Algorithm

1. Start.
2. Read the array `size`.
3. If `size <= 0`, display an error and terminate.
4. Declare the variable-length array `nums[size]`.
5. Read all array elements.
6. Pass `nums` and `size` to `Bubble_sort()`.
7. In `Bubble_sort()`:
   - Compare adjacent elements.
   - Swap them if the left element is greater than the right element.
   - Repeat until the array is sorted in ascending order.
8. Display the sorted array.
9. Pass the sorted array to `occurrence()`.
10. In `occurrence()`:
    - Read `target`.
    - Search from left to right for the first occurrence.
    - Search from right to left for the last occurrence.
    - If found, display the value and index of both occurrences.
    - Otherwise, display `-1, -1`.
11. Stop.

Test Case Table

Test	Input Array	Target	Sorted Array	Expected Result
1	`5 7 7 8 8 10`	8	`5 7 7 8 8 10`	First: 8 at index 3, Last: 8 at index 4
2	`5 7 7 8 10`	8	`5 7 7 8 10`	First: 8 at index 3, Last: 8 at index 3
3	`5 7 7 8 8 10`	6	`5 7 7 8 8 10`	`-1, -1`
4	`8 3 8 2 8 5`	8	`2 3 5 8 8 8`	First: 8 at index 3, Last: 8 at index 5
5	`4 4 4 4`	4	`4 4 4 4`	First: 4 at index 0, Last: 4 at index 3
6	`10`	10	`10`	First: 10 at index 0, Last: 10 at index 0

C Program

```
#include <stdio.h>
```

```c
/* Function to sort the array using Bubble Sort */
void Bubble_sort(int *nums, int size)
{
    int i, j, temp;

    for (i = 0; i < size - 1; i++)
    {
        for (j = 0; j < size - i - 1; j++)
        {
            if (*(nums + j) > *(nums + j + 1))
            {
                temp = *(nums + j);
                *(nums + j) = *(nums + j + 1);
                *(nums + j + 1) = temp;
            }
        }
    }
}

/* Function to find first and last occurrence */
void occurrence(int *nums, int size)
{
    int target;
    int first = -1, last = -1;
    int i;

    printf("Enter target: ");
    scanf("%d", &target);

    /* Find first occurrence */
    for (i = 0; i < size; i++)
    {
        if (*(nums + i) == target)
        {
            first = i;
            break;
        }
    }

    /* Target not found */
    if (first == -1)
    {
        printf("-1, -1\n");
        return;
```

```
    }

    /* Find last occurrence */
    for (i = size - 1; i >= 0; i--)
    {
        if (*(nums + i) == target)
        {
            last = i;
            break;
        }
    }

    printf("First occurrence: %d at index %d\n",
           *(nums + first), first);

    printf("Last occurrence: %d at index %d\n",
           *(nums + last), last);
}

int main()
{
    int size, i;

    printf("Enter the size of the array: ");
    scanf("%d", &size);

    if (size <= 0)
    {
        printf("Invalid array size.\n");
        return 0;
    }

    /* Variable-Length Array */
    int nums[size];

    printf("Enter %d elements:\n", size);

    for (i = 0; i < size; i++)
    {
        scanf("%d", &nums[i]);
    }

    /* Pass array address to Bubble_sort() */
    Bubble_sort(nums, size);
```

```
    printf("\nSorted array: ");
    for (i = 0; i < size; i++)
    {
        printf("%d ", nums[i]);
    }

    printf("\n");

    /* Pass sorted array to occurrence() */
    occurrence(nums, size);

    return 0;
}
```

Execution

